

Bangladesh University of Engineering and Technology (BUET)

Department of Computer Science and Engineering

Design Report

Tool 9: Dictionary Attack and Known Password Attack

Course: CSE406 — Computer Security Sessional

Assignment: Attack Tool 9

Group Members

Member	Student ID
Arnob Biswas	2105015
Zaki Rehnoom Unmona	2105016

Contents

1	Overview of the Project Idea	1
2	Network Topology, Components, and Timing Diagrams	1
2.1	Topology	1
2.2	Components	2
2.3	Timing Diagrams	3
2.3.1	Normal (legitimate) signup + login — browser flow	3
2.3.2	Dictionary attack — repeated exchange, one account	3
2.3.3	Known-password (spray) attack — repeated exchange, many accounts	4
2.3.4	With the lockout defense active	4
3	Packet and Frame Structure	4
3.1	Request (attacker or browser → server), one per login attempt	4
3.2	Responses, by outcome	5
3.3	Design Notes	5
4	Implementation Plan	5
5	Design Justification	6
6	Expected Outcome of a Successful Attack	7
7	Defense Idea	8

1 Overview of the Project Idea

Two related password-guessing attacks are implemented and demonstrated in this project, together with a working defense against them:

- **Dictionary attack** — *one specific account* is targeted, and a large ordered list of candidate passwords (a “dictionary” of common/likely passwords) is tried against it, one at a time, until one is accepted by the server or the list is exhausted. The attack works because passwords that appear in common wordlists are chosen by many users.
- **Known password attack** (password spraying / credential reuse) — a small set of *known* passwords is used as the starting point instead — defaults, previously breached passwords, or passwords obtained from another attack (e.g. sniffing) — and each one is tried against a *wide list of accounts*. Password reuse and default credentials are exploited by this, rather than weak individual passwords, and detection is harder with a naive per-account lockout, since many failed attempts are not received by any single account.

The target will be a vulnerable application built from scratch rather than an existing service such as SSH or FTP: a small signup/login web application (Flask + SQLite). The *victim action* for the demo will be a normal user signing up and logging in through a browser; the *attacker action* will be the two attacks above, driven by HTTP client code.

Alongside the attacks, a defense mechanism (account lockout combined with slow password hashing) will be built, so that mitigation of the attacks, not just their execution, can be shown in the report.

2 Network Topology, Components, and Timing Diagrams

2.1 Topology

The following topology will be used for development and the final demo: two VMs on a host-only virtual network, plus a legitimate browser client exercising the same webapp.

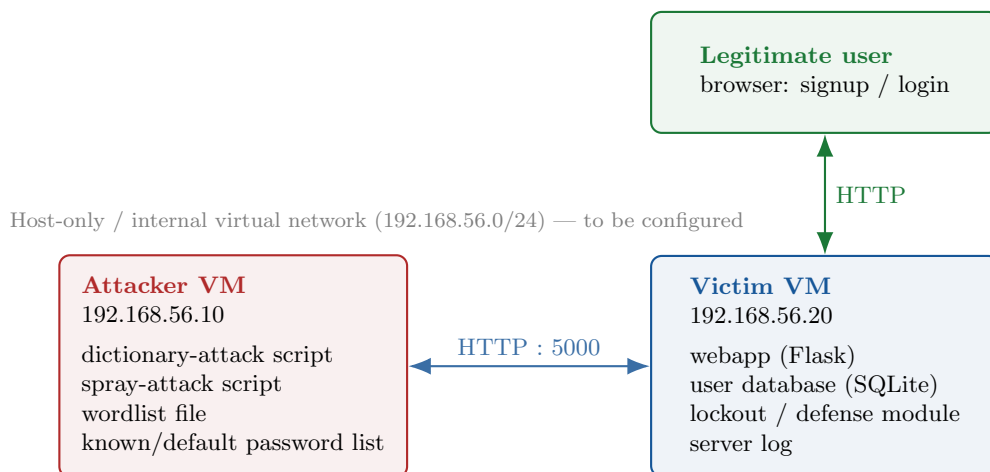


Figure 1: Attack topology: two VMs on a host-only virtual network, plus a legitimate browser client exercising the same webapp.

2.2 Components

Component	Role
Victim webapp (Flask)	Serves signup, login, and dashboard pages; the attack target
User database (SQLite)	Stores usernames, salts, and password hashes
Defense module	Tracks failed login attempts per account and enforces temporary lockout; also selects the slow-hash mode
Dictionary-attack script	Attacker tool: many candidate passwords against one account
Spray-attack script	Attacker tool: a few known passwords against many accounts
Wordlist / known-password list	Input data for the two attacker scripts
Server / attacker logs	Record every login attempt and outcome

Table 1: System components and their roles.

2.3 Timing Diagrams

The protocol to be implemented and the attack strategies to be run against it are described in the diagrams below —

2.3.1 Normal (legitimate) signup + login — browser flow

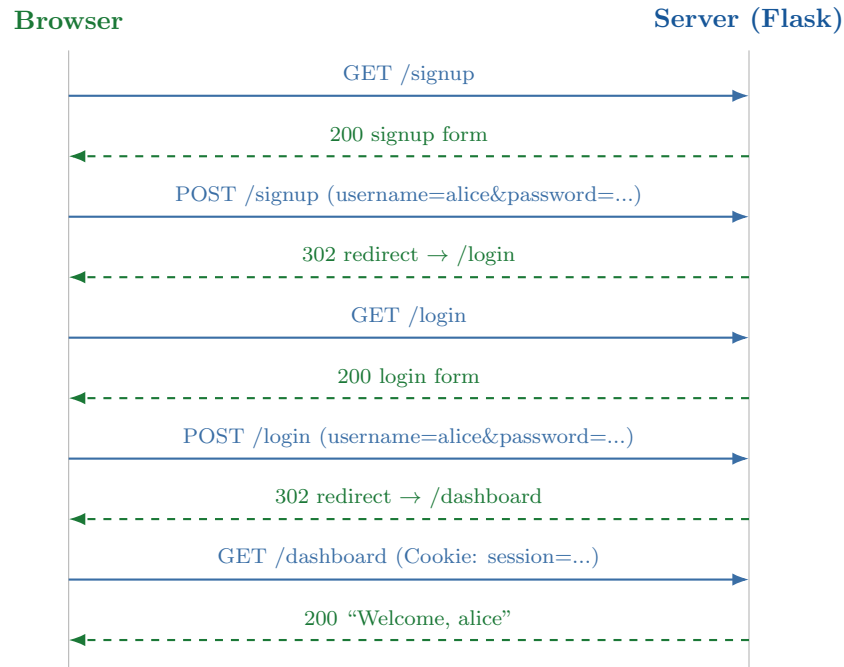


Figure 2: Normal protocol timing: one legitimate signup followed by one login.

2.3.2 Dictionary attack — repeated exchange, one account

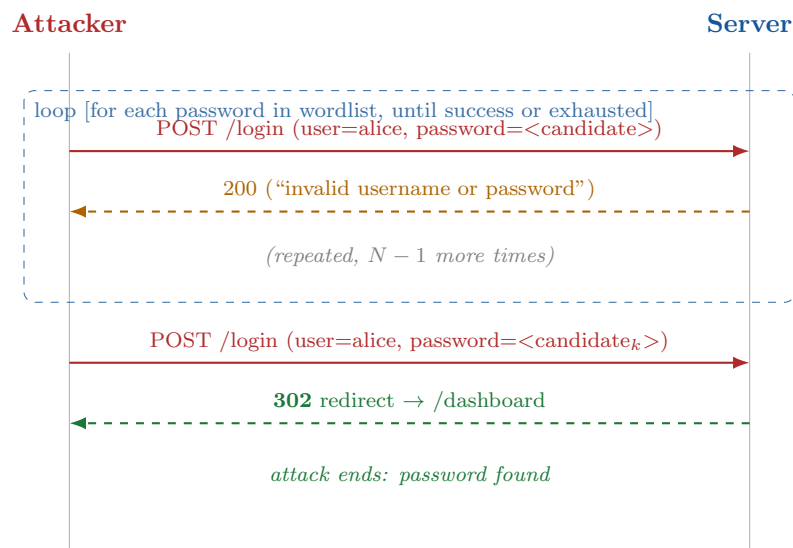


Figure 3: Dictionary attack: many candidate passwords tried in sequence against a single account.

2.3.3 Known-password (spray) attack — repeated exchange, many accounts

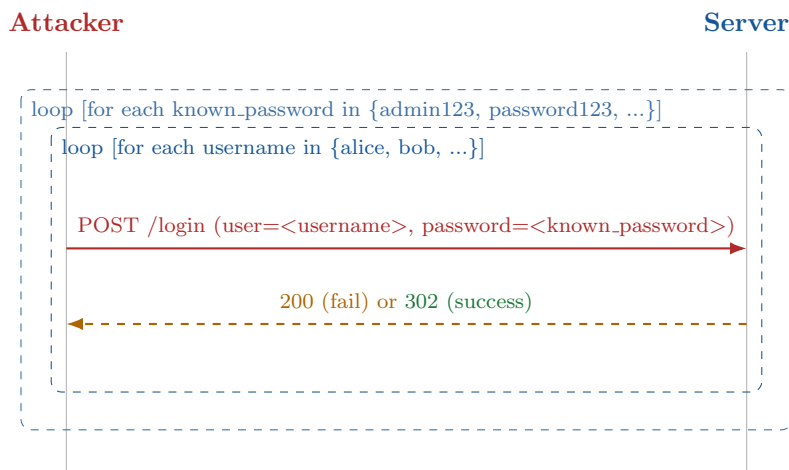


Figure 4: Known-password / spray attack: few known passwords tried across many accounts.

2.3.4 With the lockout defense active

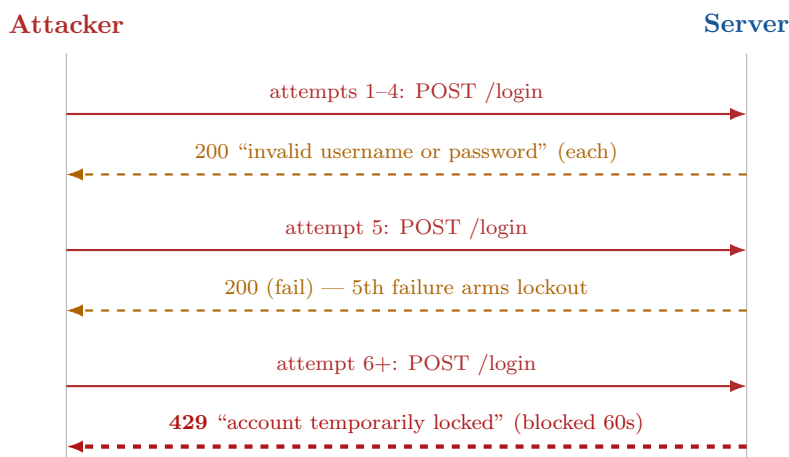


Figure 5: Lockout defense: the dictionary/spray attack should be blocked after 5 failed attempts within a 30-second window.

3 Packet and Frame Structure

No raw Ethernet/IP crafting is needed here, since the exploit is entirely at the application layer, carried over an ordinary TCP connection. What is still being designed from scratch is the *application behaviour and the payload* carried by the HTTP request, and how it is meant to be interpreted by the server — not an existing authentication protocol implementation. Transport will be standard TCP carrying HTTP/1.1 (Flask’s built-in development server); the exact request/response format below is the intended design for the login endpoint.

3.1 Request (attacker or browser → server), one per login attempt

```
POST /login HTTP/1.1
Host: 192.168.56.20:5000
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

username=alice&password=sunshine
```

3.2 Responses, by outcome

Outcome	Status	Notable headers / body
Success	302 FOUND	Location: /dashboard, Set-Cookie: session=...
Failure	200 OK	login page HTML containing “invalid username or password”
Locked (defense on)	429 TOO MANY REQUESTS	login page HTML containing “account temporarily locked”

Table 2: HTTP response codes and bodies for each login outcome.

3.3 Design Notes

- Its own TCP connection will be opened by each attempt (one HTTP request is issued per attempt by the attacker scripts, with no session reuse) — a full 3-way handshake, HTTP request/response, then FIN/ACK teardown per attempt. Since an ordinary HTTP request/response is all that is exchanged, the attacker’s own logs (timestamp, username, password tried, response) are enough to document every attempt for the Final Report.
- The response to a failed login will be the same generic “invalid username or password” whether the username does not exist or the password is wrong, so that account existence is not leaked.
- Passwords will be sent as cleartext form data over plain HTTP in this demo (no TLS), which is worth noting as a limitation versus a production deployment, which would use HTTPS.

4 Implementation Plan

The project will be built in the following stages.

1. Victim webapp (Flask + SQLite).

- Signup endpoint: a new account is created, and the password is hashed and stored.
- Login endpoint: submitted credentials are verified, and a distinct outcome is returned for success, failure, and logout (Section 3).
- Dashboard endpoint: reachable only after a successful login, so that the “victim action” has something concrete to demonstrate.

- Two selectable hashing modes: a fast single SHA-256 hash (undefended baseline) and a salted PBKDF2 hash with a high iteration count (defended mode), toggled by one configuration flag so that both can be demonstrated by the same webapp without code changes.
- 2. Seed data.** A handful of demo accounts will be created: some will be given passwords drawn from the dictionary wordlist, two will be given the same default password (for the spray attack), and one will be given a strong, list-absent password as a negative control.
 - 3. Attacker tools.**
 - Dictionary-attack script: a wordlist is iterated against one account via HTTP POST requests to `/login`, stopping on success or lockout.
 - Spray-attack script: a small known-password list is iterated against many accounts, with one password tried against all usernames before the next is used.
 - Every attempt (timestamp, username, password tried, response) is logged by both scripts, for the Final Report.
 - 4. Defense mechanism.** Failed attempts will be recorded per username by a lockout tracker, and an account will be temporarily locked once a threshold is reached, plus the slow-hash mode described above (details in Section 7).
 - 5. Test environment.** Initial development and functional testing will be carried out on a single machine (localhost), so that protocol logic is validated before network variables are introduced; the final demo will be run on two VirtualBox VMs on a host-only network, matching the topology in Section 2.
 - 6. Evaluation.** Both attacks will be run against the undefended webapp, and outcome/timing will be recorded; the defense mode will then be enabled, the account store reset, and the same attacks re-run, with the same metrics recorded; the two runs will be compared for the Final Report.

5 Design Justification

Before the evaluation in Section 6 is presented, the reasoning behind why the design above is expected to work is laid out here.

- **The dictionary attack is expected to succeed on a weak password.** Password-reuse and weak-password studies consistently find that a large share of real-world passwords are drawn from a comparatively small set of common strings, which is exactly what a dictionary attack exploits. The seeded accounts `alice`, `bob`, and `carol` will be given passwords (`sunshine`, `dragon`, `letmein`) that appear inside the 50-entry wordlist, so a match is expected to be found well before the list is exhausted. `dave`'s password, by contrast, will be high-entropy and deliberately absent from the wordlist, so it should resist the attack. This negative control is what will allow a successful crack to be attributed to the password actually being weak, rather than to a bug in the attack script.
- **The spray attack is expected to succeed independently of the dictionary.** `eve` and `frank` will be given the single shared password `admin123`, which does not need to appear anywhere in the 50-word dictionary at all. Only one correct guess per *password* is needed by the spray attack, amortized over every account it is tried against, so both accounts can be compromised in

a single pass regardless of how the dictionary attack performs. This is why the two attacks are treated as complementary rather than redundant: one exploits weak individual passwords, and the other exploits password reuse across accounts.

- **No raw packet or frame crafting should be required.** A login form is an application-layer exploit. Reliable, in-order delivery of the HTTP request is already guaranteed by TCP, so the “exploit” lives entirely in the payload and server logic (Section 3), not in headers manipulated below layer 4. This is regarded as a natural consequence of the attack class, not a shortcut being taken.
- **The logout defense is expected to stop the dictionary attack.** With the threshold set to $k = 5$ failed attempts per 30-second window, only k guesses, in expectation, will be available to an attacker who does not already know the password before a 60-second logout is triggered. Against a 50-word list, this caps the probability of success per logout cycle at $k/50 = 10\%$ for a password placed uniformly at random in the list, compared to a near-certainty, given enough time, in the undefended case. This is why the defended dictionary attack is expected to be blocked rather than merely slowed down.
- **The slow hash is expected to matter even when logout does not.** PBKDF2-HMAC-SHA256 at 200,000 iterations costs on the order of 2×10^5 times as many underlying hash operations as the single SHA-256 hash used in the undefended mode. This is the standard argument for slow hashing: the CPU cost of *every* guess is raised by several orders of magnitude, which still matters in a scenario the per-account logout cannot reach, namely an attacker who has exfiltrated the password-hash table and is guessing offline, with no server left to enforce a logout.
- **Failure responses are designed not to leak account existence.** The same generic “invalid username or password” message (Section 3) will be returned whether the password is wrong or the username does not exist at all, and hashing will be performed against a dummy salt when the username is missing, so that a nonexistent account does not respond measurably faster. Together these are meant to stop an attacker from using the login endpoint itself to enumerate valid usernames before attacking them.

6 Expected Outcome of a Successful Attack

Assuming the implementation works as intended, the following outcomes are expected.

- **Dictionary attack.** For an account whose password is a common word included in the wordlist, the attack is expected to succeed within a modest number of attempts (a small fraction of the wordlist), with a successful-login response returned and the correct password revealed. When attempted against the control account with a strong, list-absent password, the entire wordlist is expected to be exhausted without success — confirming that the tool is working correctly and is failing only because the password genuinely is not guessable this way, not because of a bug in the attack script.
- **Known-password (spray) attack.** For the two accounts seeded with the same default password, both are expected to be compromised by a single spray pass, without either password needing to appear in the larger dictionary wordlist — demonstrating that password reuse specifically is exploited by this attack, as distinct from the dictionary attack’s brute-force search.
- **Undefended timing.** Without rate limiting, logins are expected to be attempted about as fast

as the network and a single fast hash allow, so a wordlist of realistic size (on the order of 50–100 entries) is expected to be exhausted in well under a second, and the vulnerable accounts are expected to be cracked almost immediately.

- **Defended timing.** With the lockout and slow-hash defense enabled, each attempt is expected to take noticeably longer, and the dictionary attack is expected to be cut short by the lockout (after a small, fixed number of failed attempts) well before even a modest wordlist can be exhausted — i.e. a successful attack under the undefended configuration is expected to become a *blocked* attack under the defended one, given the same wordlist and target account.

7 Defense Idea

Two complementary defenses will be built, both togglable via the same “defense mode” flag on the webapp, so that the undefended and defended configurations can be compared by running the exact same attacker scripts against them.

1. **Account lockout.** Failed login attempts are tracked per username by the server. After a small threshold of failures (e.g. 5) within a short time window (e.g. 30 seconds), the account is temporarily locked (e.g. for 60 seconds), and 429 TOO MANY REQUESTS is returned for further attempts regardless of the password supplied. The number of guesses that can be made against a single account before being stopped is directly capped by this, and both the dictionary attack (many attempts, one account) and, per-account, the spray attack are blocked.
2. **Slow password hashing.** In defended mode, passwords are hashed with PBKDF2-HMAC-SHA256 using a high iteration count (e.g. 200,000) instead of a single fast SHA-256 hash. This is a standard, well-documented technique by which the CPU cost of each password check is deliberately raised by one to two orders of magnitude, and by which guessing is slowed down even in scenarios (e.g. an offline attack on a stolen hash) where a lockout counter would not apply.

	Undefended	Defended
Cost per login attempt	fast (single SHA-256 hash)	significantly higher (PBKDF2, high iteration count)
Dictionary attack outcome	expected to succeed	expected to be blocked by lockout
Spray attack outcome	expected to succeed on reused-password accounts	expected to be throttled per account

Table 3: Expected before/after effect of the lockout + slow-hash countermeasure.